Lab Report: Unsupervised Machine Learning – Clustering

Course: BFOR 516 – Advanced Data Analytics for Cybersecurity
Student: Spencer Kone
Date: February 23, 2026

AI Usage Statement

Artificial Intelligence tools were used **only** for code generation and visualization assistance. All feature selection decisions, analytical interpretations, and conclusions presented in this report were independently developed and written by the student (Spencer Kone).

---

1. Research Question

Are there linguistic patterns observable in data labeled as "Fake"?

The objective of this analysis is to identify distinct stylistic groups within fake news data. Rather than examining *what* is being said (entities or topics), this study focuses on *how* it is being said (linguistic style), using features such as word count, part-of-speech frequencies, and punctuation usage.

---

2. Feature Selection

For this clustering analysis, I selected **21 linguistic structure ("HOW") features**. These features were chosen to capture the stylistic "fingerprint" of textual delivery rather than semantic content.

Feature Categories

- **Lexical Statistics**
    - Word count
    - Average word length
    - Maximum word length
    - Minimum word length

- **Syntactic Features**
    - Frequency of present-tense verbs
    - Frequency of past-tense verbs
    - Frequency of adjectives
    - Frequency of adverbs
    - Frequency of pronouns
    - Frequency of conjunctions
- **Punctuation and Style**
    - Exclamation marks
    - Question marks
    - Capital letters
    - Digits
    - Ampersands
- **Complexity Indicators**
    - Frequency of long words
    - Frequency of short words

All features were scaled using StandardScaler to prevent variables with larger numeric ranges (e.g., word count) from disproportionately influencing the distance-based clustering algorithms.

3. Findings – Part 1: Fake Data Clustering

3.1 K-Means Clustering Results

The **Elbow Method** and **Silhouette Analysis** suggested an optimal value of **K = 3**. Three distinct linguistic styles emerged within the fake-labeled data:

Cluster 0 – "Formal Outliers" (n = 438)

- Average length: 79 words
- Average word length: 9.0
- Average capital letters: 76

This small but highly distinct group contains long segments with complex vocabulary and extreme capitalization. These likely represent extended excerpts or technical-style propaganda content.

Cluster 1 – "Brief/Minimalist" (n = 28,839)

- Average length: 24 words
- Low adjective and verb usage

This cluster consists of short, simple statements. It resembles headline-style or punch-driven fake content.

Cluster 2 – "Elaborate Sensationalist" (n = 35,991)

- Average length: 45 words
- High punctuation usage (exclamation/question marks)
- High frequency of adjectives and present-tense verbs

This cluster reflects a more emotional and sensational delivery style, commonly associated with misinformation narratives.

3.2 Hierarchical Clustering Results

Using **Ward's Linkage**, the dendrogram also indicated three primary branches. The results broadly aligned with K-Means but provided structural insight into the data hierarchy.

- The "Formal Outliers" group was separated first, confirming its substantial distance from more common short-form content.
- The two larger clusters (short vs. medium length) showed partial overlap but remained distinguishable based on complexity and stylistic markers.

3.3 Method Comparison

Metric	K-Means (K = 3)	Hierarchical (K = 3)
Silhouette Score	0.1620	0.0990
Stability	High	Medium (sample-dependent)

Conclusion:
K-Means produced tighter and more mathematically distinct clusters for this high-dimensional linguistic dataset. However, Hierarchical clustering provided valuable visualization of how the "Formal Outlier" group is fundamentally separated from other stylistic patterns.

---

4. Findings – Part 2 (Practice Task): Tweet Linguistic Patterns

The clustering analysis was repeated on the full dataset (including both “True” and “Fake” labels) to determine whether fake tweets naturally cluster together.

4.1 Cluster Characteristics

K-Means identified **K = 2** as optimal:

Cluster 0 (52% Fake) – “Intensive Style”

- Average length: 45 words
- Higher capitalization (14.6)
- Greater adjective and punctuation frequency

Cluster 1 (44% Fake) – “Simple Style”

- Average length: 23 words
- Lower grammatical and stylistic complexity

4.2 Interpretation

The results suggest that misinformation is more likely to fall into the “Intensive” linguistic cluster. In this dataset, fake news segments tend to be slightly longer and contain more stylistic markers (punctuation, capitalization, descriptive language) than true news segments.

Although fake content is present in both clusters, the higher concentration in Cluster 0 indicates that a more verbose and emotionally intense delivery style is a common linguistic pattern within this collection.

---

5. Overall Conclusion

The unsupervised clustering analysis reveals three primary linguistic signatures within fake news data:

1. **Mass-Market Sensationalism** – Adjective-heavy, punctuated, emotionally intense content.
2. **Short-Form Misinformation** – Minimalist, headline-style statements.
3. **Technical/Propaganda Outliers** – Highly complex, long-form segments with elevated structural density.

Both K-Means and Hierarchical clustering methods identified consistent stylistic groupings. However, K-Means demonstrated stronger mathematical separation for large-scale, high-dimensional feature sets.

Overall, the findings suggest that truthfulness may be partially signaled by the **intensity and structural complexity** of delivery style. While clustering alone does not determine veracity, linguistic style appears to provide meaningful unsupervised signals for distinguishing patterns within fake-labeled data.

Week 4 Lab: Unsupervised Learning - Clustering

BFOR516 - Advanced Data Analytics for Cyber

Student: Spencer Kone

Date: February 23, 2026

AI Usage Statement:

- AI tools were used ONLY for code generation assistance.
- All interpretation, results analysis, and conclusions are from
  the student (Spencer Kone).

Research Question:

Are there linguistic patterns that we can observe in data labeled as 'Fake'?

Practice Question:

Are there linguistic patterns unique to tweets labeled as 'Fake'?

"""

```
import pandas as pd
import numpy as np
import sys
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch
```

```
def log(msg):
    print(msg)
    sys.stdout.flush()

# ==========================================================================
# 1. LOAD AND PREPARE DATA
# ==========================================================================
log("Loading dataset...")
df = pd.read_csv('Truth_Seeker_Dataset.csv')
log(f"Dataset shape: {df.shape}")
log(f"Columns: {list(df.columns)}")

# Target column: BinaryNumTarget (1.0 = True, 0.0 = Fake)
target = 'BinaryNumTarget'
df = df.dropna(subset=[target])

log(f"\nTarget distribution:")
log(f"  True (1.0):  {(df[target] == 1.0).sum()}")
log(f"  Fake (0.0):  {(df[target] == 0.0).sum()}")
```

```
# ==========================================================================
# 2. FEATURE SELECTION - Linguistic Structure (HOW) Features
# ==========================================================================
# Selecting linguistic/stylistic features that describe HOW something is said
# (not WHAT is said). These capture grammar, punctuation, complexity, tone.
how_features = [
    'Word count', 'Max word length', 'Min word length', 'Average word length',
    'present_verbs', 'past_verbs', 'adjectives', 'adverbs', 'adpositions',
    'pronouns', 'TOs', 'determiners', 'conjunctions', 'dots', 'exclamation',
    'questions', 'ampersand', 'capitals', 'digits', 'long_word_freq', 'short_word_freq'
]

log(f"\nSelected HOW (Linguistic Structure) Features ({len(how_features)}):")
for f in how_features:
    log(f"  - {f}")

# ==========================================================================
# 3. FILTER TO FAKE DATA ONLY & PREPARE
# ==========================================================================
log("\n" + "="*70)
log("PART 1: Clustering linguistic patterns in FAKE-labeled data")
log("="*70)
```

```
df_fake = df[df[target] == 0.0].copy()
log(f"\nFake data subset: {df_fake.shape[0]} rows")

# Drop any rows with NaN in our features
df_fake = df_fake.dropna(subset=how_features)
log(f"After dropping NaN: {df_fake.shape[0]} rows")

# Extract & scale features
X_fake = df_fake[how_features].fillna(0)
scaler = StandardScaler()
X_fake_scaled = scaler.fit_transform(X_fake)
log("Features scaled using StandardScaler.")

# ==========================================================================
# 4. K-MEANS CLUSTERING ON FAKE DATA
# ==========================================================================
log("\n--- K-Means Clustering (Fake Data) ---")

# 4a. Elbow Method to find optimal K
inertia = []
sil_scores = []
k_range = range(2, 11)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_fake_scaled)
```

```
    inertia.append(kmeans.inertia_)
    sil = silhouette_score(X_fake_scaled, labels, sample_size=10000, random_state=42)
    sil_scores.append(sil)
    log(f"  K={k}: Inertia={kmeans.inertia_:.2f}, Silhouette={sil:.4f}")

# Elbow plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(k_range, inertia, 'bx-')
ax1.set_xlabel('Number of Clusters (k)')
ax1.set_ylabel('Inertia')
ax1.set_title('Elbow Method - Fake Data (Linguistic Features)')
ax1.grid(True, alpha=0.3)

ax2.plot(k_range, sil_scores, 'rx-')
ax2.set_xlabel('Number of Clusters (k)')
ax2.set_ylabel('Silhouette Score')
ax2.set_title('Silhouette Score - Fake Data (Linguistic Features)')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fake_kmeans_elbow.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: fake_kmeans_elbow.png")

# 4b. Run K-Means with best K (use K with highest silhouette)
best_k_idx = np.argmax(sil_scores)
best_k = list(k_range)[best_k_idx]
log(f"\nBest K by Silhouette Score: {best_k} (score={sil_scores[best_k_idx]:.4f})")
```

```
kmeans_best = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df_fake['KMeans_Cluster'] = kmeans_best.fit_predict(X_fake_scaled)

# 4c. Analyze K-Means clusters
log(f"\nK-Means Cluster Analysis (K={best_k}):")
km_analysis = df_fake.groupby('KMeans_Cluster')[how_features].agg(['mean'])
km_counts = df_fake.groupby('KMeans_Cluster').size().reset_index(name='count')
log(km_counts.to_string(index=False))

# Print key feature means per cluster
log("\nKey feature means per K-Means cluster:")
key_feats = ['Word count', 'Average word length', 'exclamation', 'questions',
             'capitals', 'present_verbs', 'past_verbs', 'adjectives',
             'long_word_freq', 'short_word_freq']
summary = df_fake.groupby('KMeans_Cluster')[key_feats].mean()
log(summary.to_string())

# 4d. K-Means scatter plot
plt.figure(figsize=(10, 6))
for c in range(best_k):
    mask = df_fake['KMeans_Cluster'] == c
    plt.scatter(
        df_fake.loc[mask, 'Word count'],
        df_fake.loc[mask, 'Average word length'],
        label=f'Cluster {c} (n={mask.sum()})',
        alpha=0.5, s=10
    )
```

```
plt.xlabel('Word Count')
plt.ylabel('Average Word Length')
plt.title(f'K-Means Clusters (K={best_k}) - Fake Data Linguistic Features')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('fake_kmeans_clusters.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: fake_kmeans_clusters.png")

# ==========================================================================
# 5. HIERARCHICAL CLUSTERING ON FAKE DATA
# ==========================================================================
log("\n--- Hierarchical Clustering (Fake Data) ---")

# 5a. Sample for dendrogram (full dataset too large for linkage)
sample_size = min(10000, len(df_fake))
df_fake_sample = df_fake.sample(n=sample_size, random_state=42)
X_fake_sample_scaled = scaler.fit_transform(df_fake_sample[how_features].fillna(0))
log(f"Sampled {sample_size} rows for hierarchical clustering.")

# 5b. Dendrogram
plt.figure(figsize=(15, 7))
plt.title('Dendrogram - Fake Data (Linguistic Features)')
dendrogram = sch.dendrogram(sch.linkage(X_fake_sample_scaled, method='ward'))
plt.xlabel('Data Points')
```

```
plt.ylabel('Euclidean Distance')
plt.savefig('fake_hierarchical_dendrogram.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: fake_hierarchical_dendrogram.png")

# 5c. Merge distance plot
Z = sch.linkage(X_fake_sample_scaled, method='ward')
distances = sorted(Z[:, 2], reverse=True)
plt.figure(figsize=(10, 5))
plt.plot(range(1, min(51, len(distances)+1)), distances[:50], 'o-')
plt.title('Merge Distances - Fake Data (Hierarchical Clustering)')
plt.xlabel('Number of Merges')
plt.ylabel('Merge Distance')
plt.grid(True, alpha=0.3)
plt.savefig('fake_hierarchical_merge_distance.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: fake_hierarchical_merge_distance.png")

# 5d. Run Agglomerative Clustering with same K as K-Means for comparison
hc = AgglomerativeClustering(n_clusters=best_k, linkage='ward')
df_fake_sample['HC_Cluster'] = hc.fit_predict(X_fake_sample_scaled)

log(f"\nHierarchical Cluster Analysis (K={best_k}):")
hc_counts = df_fake_sample.groupby('HC_Cluster').size().reset_index(name='count')
log(hc_counts.to_string(index=False))

log("\nKey feature means per Hierarchical cluster:")
hc_summary = df_fake_sample.groupby('HC_Cluster')[key_feats].mean()
```

```
log(hc_summary.to_string())



# 5e. Hierarchical scatter plot
plt.figure(figsize=(10, 6))
for c in range(best_k):
    mask = df_fake_sample['HC_Cluster'] == c
    plt.scatter(
        df_fake_sample.loc[mask, 'Word count'],
        df_fake_sample.loc[mask, 'Average word length'],
        label=f'Cluster {c} (n={mask.sum()})',
        alpha=0.5, s=10
    )
plt.xlabel('Word Count')
plt.ylabel('Average Word Length')
plt.title(f'Hierarchical Clusters (K={best_k}) - Fake Data Linguistic Features')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('fake_hierarchical_clusters.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: fake_hierarchical_clusters.png")
```

```
#
==========================================================================
=======
                    # 6. COMPARISON: K-Means vs Hierarchical on Fake Data
#
==========================================================================
=======
log("\n--- K-Means vs Hierarchical Comparison (Fake Data) ---")

# Run KMeans on same sample for apples-to-apples
km_sample = KMeans(n_clusters=best_k, random_state=42, n_init=10)
df_fake_sample['KM_Cluster_Sample'] = km_sample.fit_predict(X_fake_sample_scaled)

km_sil = silhouette_score(X_fake_sample_scaled,
df_fake_sample['KM_Cluster_Sample'])
hc_sil = silhouette_score(X_fake_sample_scaled, df_fake_sample['HC_Cluster'])
log(f"K-Means Silhouette Score:     {km_sil:.4f}")
log(f"Hierarchical Silhouette Score:  {hc_sil:.4f}")

# Side-by-side cluster size comparison
km_sizes = df_fake_sample.groupby('KM_Cluster_Sample').size().values
hc_sizes = df_fake_sample.groupby('HC_Cluster').size().values
log(f"\nK-Means cluster sizes:     {sorted(km_sizes)}")
log(f"Hierarchical cluster sizes: {sorted(hc_sizes)}")

# Side-by-side scatter
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
for c in range(best_k):
    mask = df_fake_sample['KM_Cluster_Sample'] == c
```

```
    ax1.scatter(df_fake_sample.loc[mask, 'Word count'],
                df_fake_sample.loc[mask, 'Average word length'],
                label=f'Cluster {c}', alpha=0.5, s=10)
ax1.set_xlabel('Word Count')
ax1.set_ylabel('Average Word Length')
ax1.set_title(f'K-Means (K={best_k}) - Fake Data')
ax1.legend()
ax1.grid(True, alpha=0.3)

for c in range(best_k):
    mask = df_fake_sample['HC_Cluster'] == c
    ax2.scatter(df_fake_sample.loc[mask, 'Word count'],
                df_fake_sample.loc[mask, 'Average word length'],
                label=f'Cluster {c}', alpha=0.5, s=10)
ax2.set_xlabel('Word Count')
ax2.set_ylabel('Average Word Length')
ax2.set_title(f'Hierarchical (K={best_k}) - Fake Data')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fake_comparison_kmeans_vs_hierarchical.png', dpi=150,
bbox_inches='tight')
plt.close()
log("Saved: fake_comparison_kmeans_vs_hierarchical.png")

#
==========================================================================
========
```

```
# 7. PRACTICE: Repeat analysis for TWEET data
# ==========================================================================
========
log("\n" + "="*70)
log("PART 2 (Practice): Clustering linguistic patterns in tweets labeled 'Fake'")
log("="*70)

# The tweets have the same linguistic features since those columns describe the
# tweet text itself. We now run the exact same pipeline but use the full data
# with both label types and compare the results, focusing on Fake tweets.

# Re-load fresh for tweet analysis
df_all = pd.read_csv('Truth_Seeker_Dataset.csv')
df_all = df_all.dropna(subset=[target] + how_features)
log(f"Full dataset (after dropping NaN): {df_all.shape[0]} rows")

# Scale
X_all = df_all[how_features].fillna(0)
scaler_all = StandardScaler()
X_all_scaled = scaler_all.fit_transform(X_all)

# K-Means on all data
log("\n--- K-Means on ALL data (Fake + True) ---")
inertia_all = []
sil_all = []
for k in k_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_all_scaled)
```

```
    inertia_all.append(km.inertia_)
    sil = silhouette_score(X_all_scaled, labels, sample_size=10000, random_state=42)
    sil_all.append(sil)
    log(f"  K={k}: Inertia={km.inertia_:.2f}, Silhouette={sil:.4f}")

best_k_all_idx = np.argmax(sil_all)
best_k_all = list(k_range)[best_k_all_idx]
log(f"\nBest K for all data: {best_k_all} (Silhouette={sil_all[best_k_all_idx]:.4f})")

km_all = KMeans(n_clusters=best_k_all, random_state=42, n_init=10)
df_all['KMeans_Cluster'] = km_all.fit_predict(X_all_scaled)

# Elbow plot for all data
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(k_range, inertia_all, 'bx-')
ax1.set_xlabel('Number of Clusters (k)')
ax1.set_ylabel('Inertia')
ax1.set_title('Elbow Method - All Data (Linguistic Features)')
ax1.grid(True, alpha=0.3)

ax2.plot(k_range, sil_all, 'rx-')
ax2.set_xlabel('Number of Clusters (k)')
ax2.set_ylabel('Silhouette Score')
ax2.set_title('Silhouette Score - All Data (Linguistic Features)')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('all_kmeans_elbow.png', dpi=150, bbox_inches='tight')
```

```
plt.close()
log("Saved: all_kmeans_elbow.png")

# Analyze: what % of each cluster is Fake?
log(f"\nK-Means cluster composition (All Data, K={best_k_all}):")
cluster_comp = df_all.groupby('KMeans_Cluster').agg(
    count=('BinaryNumTarget', 'count'),
    mean_target=('BinaryNumTarget', 'mean')
)
cluster_comp['FakeNewsPropensity'] = 1 - cluster_comp['mean_target']
log(cluster_comp.to_string())

log("\nKey linguistic features per cluster (All Data):")
all_summary = df_all.groupby('KMeans_Cluster')[key_feats].mean()
log(all_summary.to_string())

# Scatter: color by cluster, marker by label
plt.figure(figsize=(12, 7))
for c in range(best_k_all):
    mask = df_all['KMeans_Cluster'] == c
    plt.scatter(
        df_all.loc[mask, 'Word count'],
        df_all.loc[mask, 'Average word length'],
        label=f'Cluster {c} (n={mask.sum()},
Fake%={cluster_comp.loc[c,"FakeNewsPropensity"]*100:.1f}%)',
        alpha=0.3, s=8
    )
plt.xlabel('Word Count')
```

```
plt.ylabel('Average Word Length')
plt.title(f'K-Means (K={best_k_all}) - All Data (Fake + True)')
plt.legend(fontsize=8)
plt.grid(True, alpha=0.3)
plt.savefig('all_kmeans_clusters.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: all_kmeans_clusters.png")

# Hierarchical on sampled all data
log("\n--- Hierarchical on ALL data ---")
sample_all_size = min(10000, len(df_all))
df_all_sample = df_all.sample(n=sample_all_size, random_state=42)
X_all_sample_scaled = scaler_all.fit_transform(df_all_sample[how_features].fillna(0))

hc_all = AgglomerativeClustering(n_clusters=best_k_all, linkage='ward')
df_all_sample['HC_Cluster'] = hc_all.fit_predict(X_all_sample_scaled)

# Dendrogram
plt.figure(figsize=(15, 7))
plt.title('Dendrogram - All Data (Linguistic Features)')
sch.dendrogram(sch.linkage(X_all_sample_scaled, method='ward'))
plt.xlabel('Data Points')
plt.ylabel('Euclidean Distance')
plt.savefig('all_hierarchical_dendrogram.png', dpi=150, bbox_inches='tight')
plt.close()
log("Saved: all_hierarchical_dendrogram.png")
```

```
# Hierarchical composition
hc_comp = df_all_sample.groupby('HC_Cluster').agg(
    count=('BinaryNumTarget', 'count'),
    mean_target=('BinaryNumTarget', 'mean')
)
hc_comp['FakeNewsPropensity'] = 1 - hc_comp['mean_target']
log(f"\nHierarchical cluster composition (All Data, K={best_k_all}):")
log(hc_comp.to_string())

log("\nKey linguistic features per HC cluster (All Data):")
hc_all_summary = df_all_sample.groupby('HC_Cluster')[key_feats].mean()
log(hc_all_summary.to_string())

# Comparison on all data sample
km_s = KMeans(n_clusters=best_k_all, random_state=42, n_init=10)
df_all_sample['KM_Cluster_Sample'] = km_s.fit_predict(X_all_sample_scaled)

km_sil_all = silhouette_score(X_all_sample_scaled,
df_all_sample['KM_Cluster_Sample'])
hc_sil_all = silhouette_score(X_all_sample_scaled, df_all_sample['HC_Cluster'])
log(f"\nAll Data Comparison:")
log(f"K-Means Silhouette Score:     {km_sil_all:.4f}")
log(f"Hierarchical Silhouette Score:  {hc_sil_all:.4f}")

log("\n" + "="*70)
log("ANALYSIS COMPLETE")
log("="*70)
log("All plots saved. See Lab Report for interpretation.")
```

Loading dataset...

Dataset shape: (134198, 64)

Columns: ['Unnamed: 0', 'majority_target', 'statement', 'BinaryNumTarget', 'tweet', 'followers_count', 'friends_count', 'favourites_count', 'statuses_count', 'listed_count', 'following', 'embeddings', 'BotScore', 'BotScoreBinary', 'cred', 'normalize_influence', 'mentions', 'quotes', 'replies', 'retweets', 'favourites', 'hashtags', 'URLs', 'unique_count', 'total_count', 'ORG_percentage', 'NORP_percentage', 'GPE_percentage', 'PERSON_percentage', 'MONEY_percentage', 'DATE_percentage', 'CARDINAL_percentage', 'PERCENT_percentage', 'ORDINAL_percentage', 'FAC_percentage', 'LAW_percentage', 'PRODUCT_percentage', 'EVENT_percentage', 'TIME_percentage', 'LOC_percentage', 'WORK_OF_ART_percentage', 'QUANTITY_percentage', 'LANGUAGE_percentage', 'Word count', 'Max word length', 'Min word length', 'Average word length', 'present_verbs', 'past_verbs', 'adjectives', 'adverbs', 'adpositions', 'pronouns', 'TOs', 'determiners', 'conjunctions', 'dots', 'exclamation', 'questions', 'ampersand', 'capitals', 'digits', 'long_word_freq', 'short_word_freq']

Target distribution:

True (1.0): 68930

Fake (0.0): 65268

Selected HOW (Linguistic Structure) Features (21):

- Word count
- Max word length
- Min word length
- Average word length
- present_verbs
- past_verbs
- adjectives
- adverbs
- adpositions
- pronouns
- TOs

- determiners
- conjunctions
- dots
- exclamation
- questions
- ampersand
- capitals
- digits
- long_word_freq
- short_word_freq

==========

PART 1: Clustering linguistic patterns in FAKE-labeled data

==========

Fake data subset: 65268 rows

After dropping NaN: 65268 rows

Features scaled using StandardScaler.

--- K-Means Clustering (Fake Data) ---

C:\Users\intel\AppData\Roaming\Python\Python313\site-packages\joblib\externals\loky\backend\context.py:131: UserWarning: Could not find the number of physical cores for the following reason:

[WinError 2] Le fichier spcifi est introuvable

Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.

  warnings.warn(

  File "C:\Users\intel\AppData\Roaming\Python\Python313\site-packages\joblib\externals\loky\backend\context.py", line 247, in _count_physical_cores

```
  cpu_count_physical = _count_physical_cores_win32()
 File "C:\Users\intel\AppData\Roaming\Python\Python313\site-packages\joblib\externals\loky\backend\context.py", line 299, in _count_physical_cores_win32
  cpu_info = subprocess.run(
    "wmic CPU Get NumberOfCores /Format:csv".split(),
    capture_output=True,
    text=True,
  )
 File "C:\Python313\Lib\subprocess.py", line 554, in run
  with Popen(*popenargs, **kwargs) as process:
     ~~~~~^^^^^^^^^^^^^^^^^^^^^^
 File "C:\Python313\Lib\subprocess.py", line 1039, in __init__
  self._execute_child(args, executable, preexec_fn, close_fds,
  ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
              pass_fds, cwd, env,
              ^^^^^^^^^^^^^^^^^^^
  ...<5 lines>...
              gid, gids, uid, umask,
              ^^^^^^^^^^^^^^^^^^^^^^
              start_new_session, process_group)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
 File "C:\Python313\Lib\subprocess.py", line 1554, in _execute_child
  hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
             ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                  # no special security
                  ^^^^^^^^^^^^^^^^^^^^^
  ...<4 lines>...
                  cwd,
```

```
                    ^^^^
                    startupinfo)
                    ^^^^^^^^^^^^
  K=2: Inertia=1164169.22, Silhouette=0.1546
  K=3: Inertia=1076836.37, Silhouette=0.1620
  K=4: Inertia=1020523.70, Silhouette=0.1217
  K=5: Inertia=975386.85, Silhouette=0.1226
  K=6: Inertia=935605.65, Silhouette=0.1259
  K=7: Inertia=899290.24, Silhouette=0.1229
  K=8: Inertia=868065.99, Silhouette=0.0897
  K=9: Inertia=842193.99, Silhouette=0.0932
  K=10: Inertia=821046.77, Silhouette=0.0873
Saved: fake_kmeans_elbow.png

Best K by Silhouette Score: 3 (score=0.1620)

K-Means Cluster Analysis (K=3):
 KMeans_Cluster  count
           0    438
           1  28839
           2  35991

Key feature means per K-Means cluster:
             Word count  Average word length  exclamation  questions   capitals
present_verbs  past_verbs  adjectives  long_word_freq  short_word_freq
KMeans_Cluster
0            79.205479          9.089162     0.203196   0.312785  76.401826      3.095890
3.851598    4.349315      33.289954       21.255708
```

1 23.932071 5.186687 0.283921 0.263289 12.814071 1.426298 1.212802 1.974964 1.685322 14.013558

2 45.496596 4.832906 0.355005 0.406074 14.443833 3.618349 2.566419 3.669112 2.383124 28.953238

Saved: fake_kmeans_clusters.png

--- Hierarchical Clustering (Fake Data) ---

Sampled 10000 rows for hierarchical clustering.

Saved: fake_hierarchical_dendrogram.png

Saved: fake_hierarchical_merge_distance.png

Hierarchical Cluster Analysis (K=3):

HC_Cluster count

0 6983

1 2953

2 64

Key feature means per Hierarchical cluster:

Word count Average word length exclamation questions capitals present_verbs past_verbs adjectives long_word_freq short_word_freq

HC_Cluster

0 42.142059 4.909296 0.410998 0.419734 16.034656 3.184018 2.337534 3.356007 2.353000 26.457397

1 20.714189 5.203202 0.091771 0.180494 8.301727 1.339316 1.060278 1.801219 1.424653 12.108026

2 87.390625 8.919440 0.171875 0.468750 79.000000 3.625000 4.281250 4.843750 35.843750 24.046875

Saved: fake_hierarchical_clusters.png

--- K-Means vs Hierarchical Comparison (Fake Data) ---

```
K-Means Silhouette Score:     0.1599
Hierarchical Silhouette Score:  0.0990

K-Means cluster sizes:     [np.int64(76), np.int64(4466), np.int64(5458)]
Hierarchical cluster sizes: [np.int64(64), np.int64(2953), np.int64(6983)]
Saved: fake_comparison_kmeans_vs_hierarchical.png

======================================================================
              PART 2 (Practice): Clustering linguistic patterns in tweets labeled 'Fake'
======================================================================
Full dataset (after dropping NaN): 134198 rows

--- K-Means on ALL data (Fake + True) ---
  K=2: Inertia=2380975.13, Silhouette=0.1674
  K=3: Inertia=2256941.24, Silhouette=0.1474
  K=4: Inertia=2110917.60, Silhouette=0.1393
  K=5: Inertia=1983268.36, Silhouette=0.1395
  K=6: Inertia=1896175.04, Silhouette=0.1373
  K=7: Inertia=1829431.29, Silhouette=0.0818
  K=8: Inertia=1751560.26, Silhouette=0.0889
  K=9: Inertia=1678250.88, Silhouette=0.0920
  K=10: Inertia=1625258.18, Silhouette=0.0955

Best K for all data: 2 (Silhouette=0.1674)
Saved: all_kmeans_elbow.png

K-Means cluster composition (All Data, K=2):
             count  mean_target  FakeNewsPropensity
```

KMeans_Cluster

0 72028 0.479522 0.520478

1 62170 0.553177 0.446823

Key linguistic features per cluster (All Data):

Word count Average word length exclamation questions capitals present_verbs past_verbs adjectives long_word_freq short_word_freq

KMeans_Cluster

0 45.540484 4.940341 0.297967 0.372647 14.629061 3.425043 2.423669 3.906620 2.815572 28.327414

1 22.955863 5.191397 0.214734 0.231269 10.749783 1.325430 1.103796 2.023677 1.593791 13.457584

Saved: all_kmeans_clusters.png

--- Hierarchical on ALL data ---

Saved: all_hierarchical_dendrogram.png

Hierarchical cluster composition (All Data, K=2):

count mean_target FakeNewsPropensity

HC_Cluster

0 6668 0.491752 0.508248

1 3332 0.570828 0.429172

Key linguistic features per HC cluster (All Data):

Word count Average word length exclamation questions capitals present_verbs past_verbs adjectives long_word_freq short_word_freq

HC_Cluster

0 42.303239 5.007425 0.324235 0.357678 15.146821 3.032693 2.209358 3.630024 2.781044 25.991452

1 20.726891 5.182305 0.151261 0.229292 8.281212 1.217887 0.997899 1.820528 1.338836 12.284214

All Data Comparison:

K-Means Silhouette Score: 0.1599

Hierarchical Silhouette Score: 0.1075

==========

ANALYSIS COMPLETE

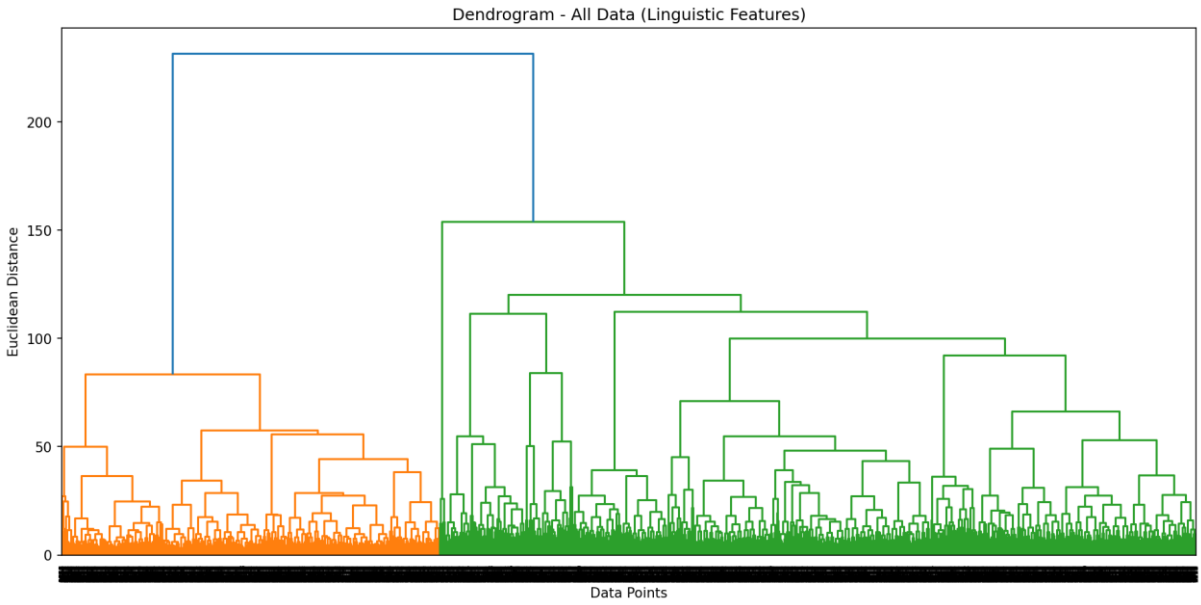
==========

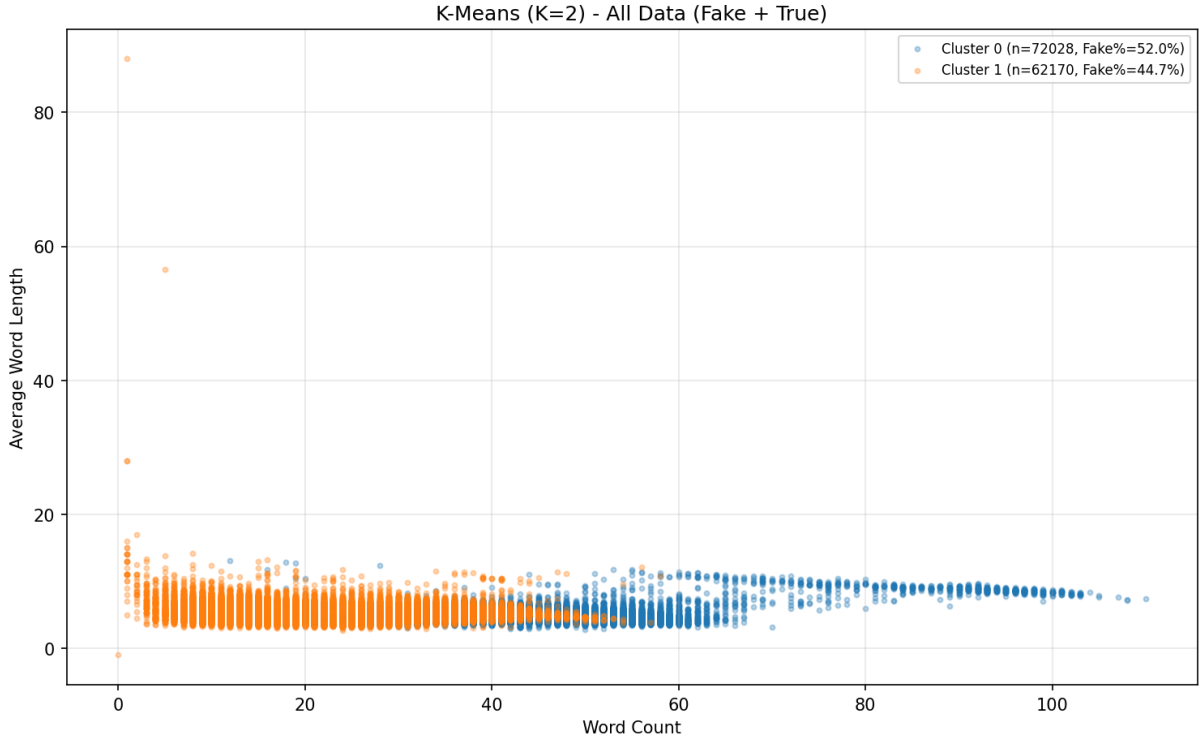
All plots saved. See Lab Report for interpretation.

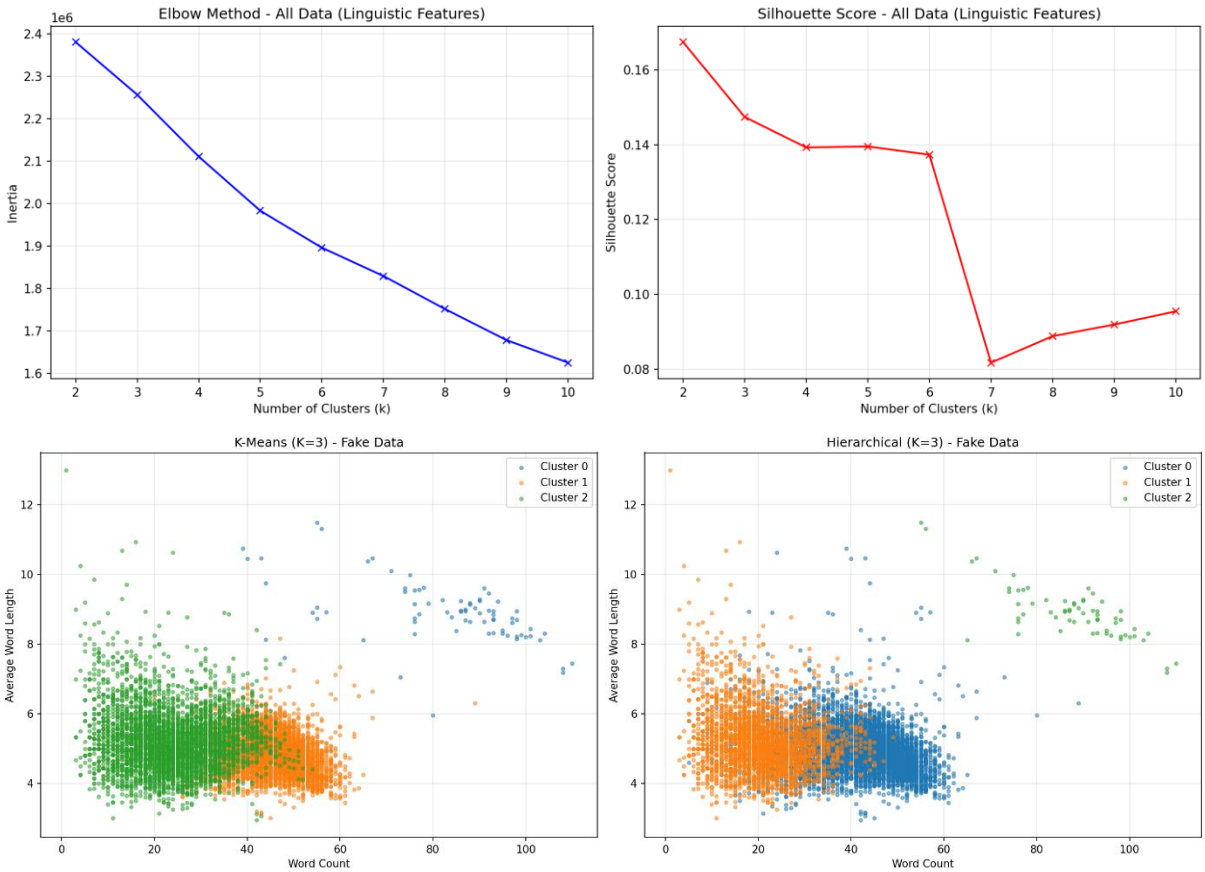
==========

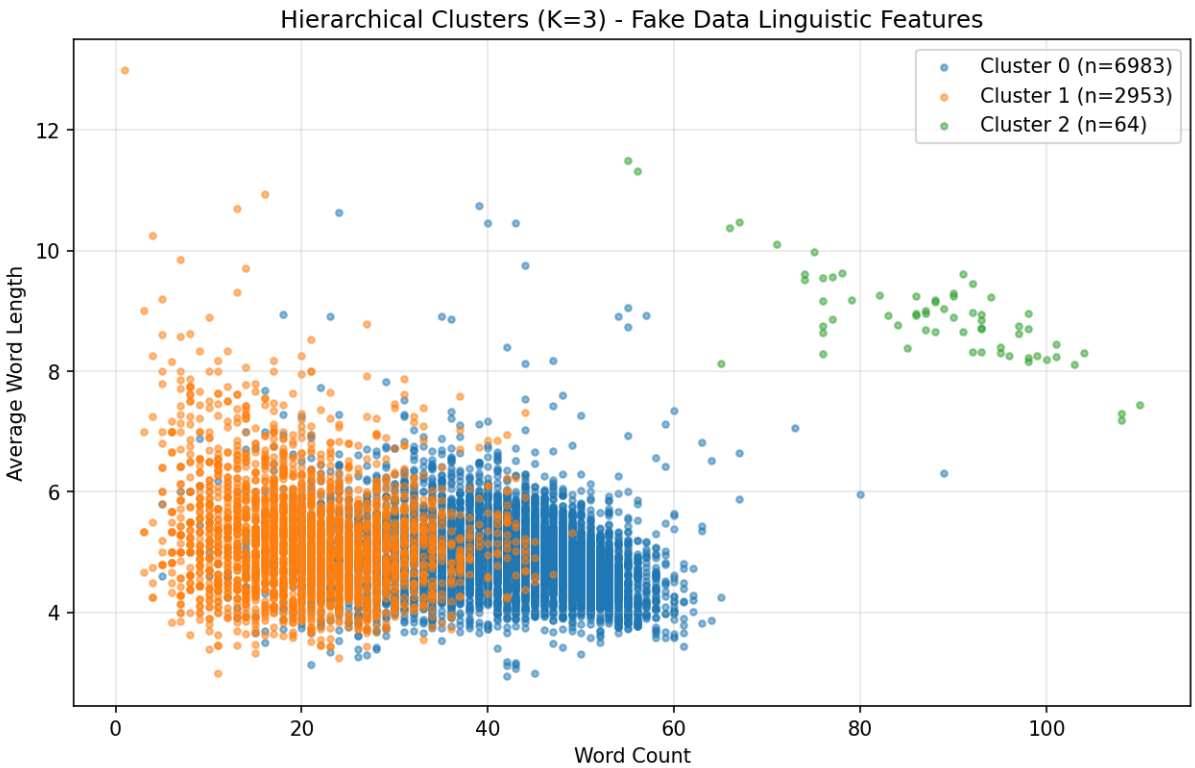
VISUALIZATION

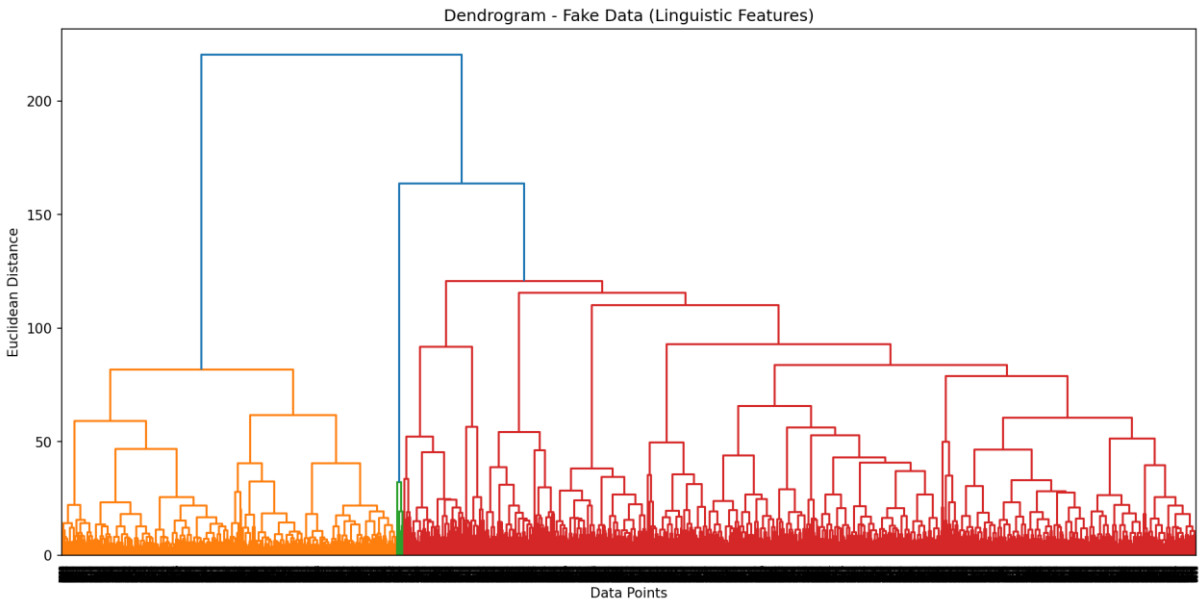
==========

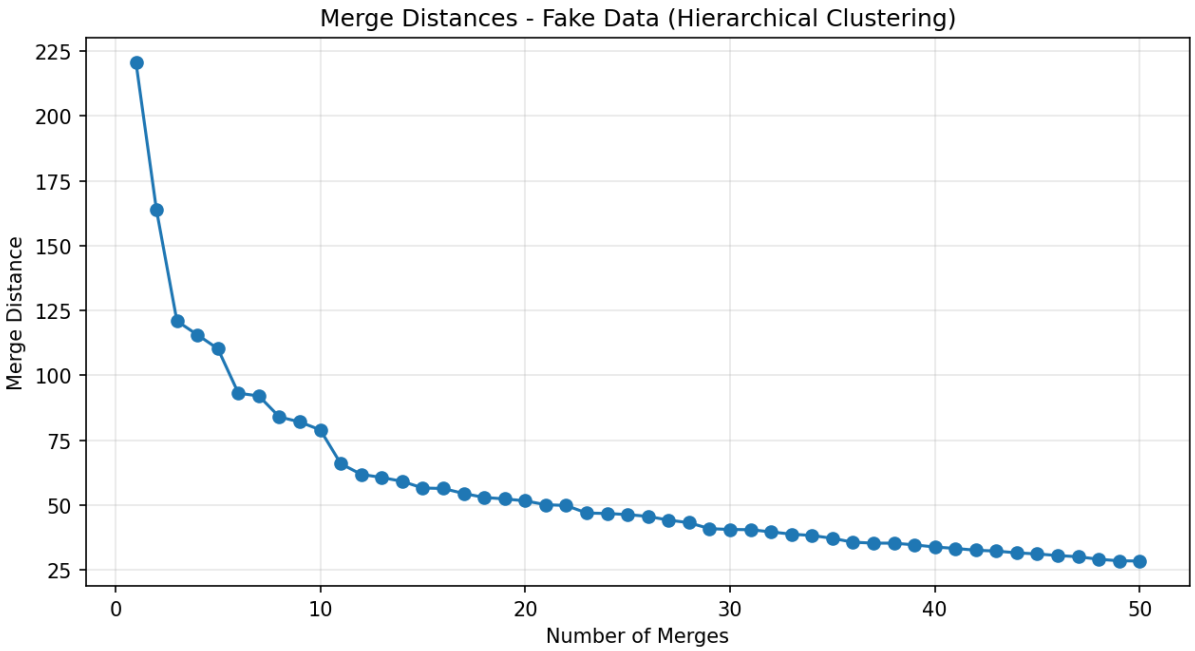


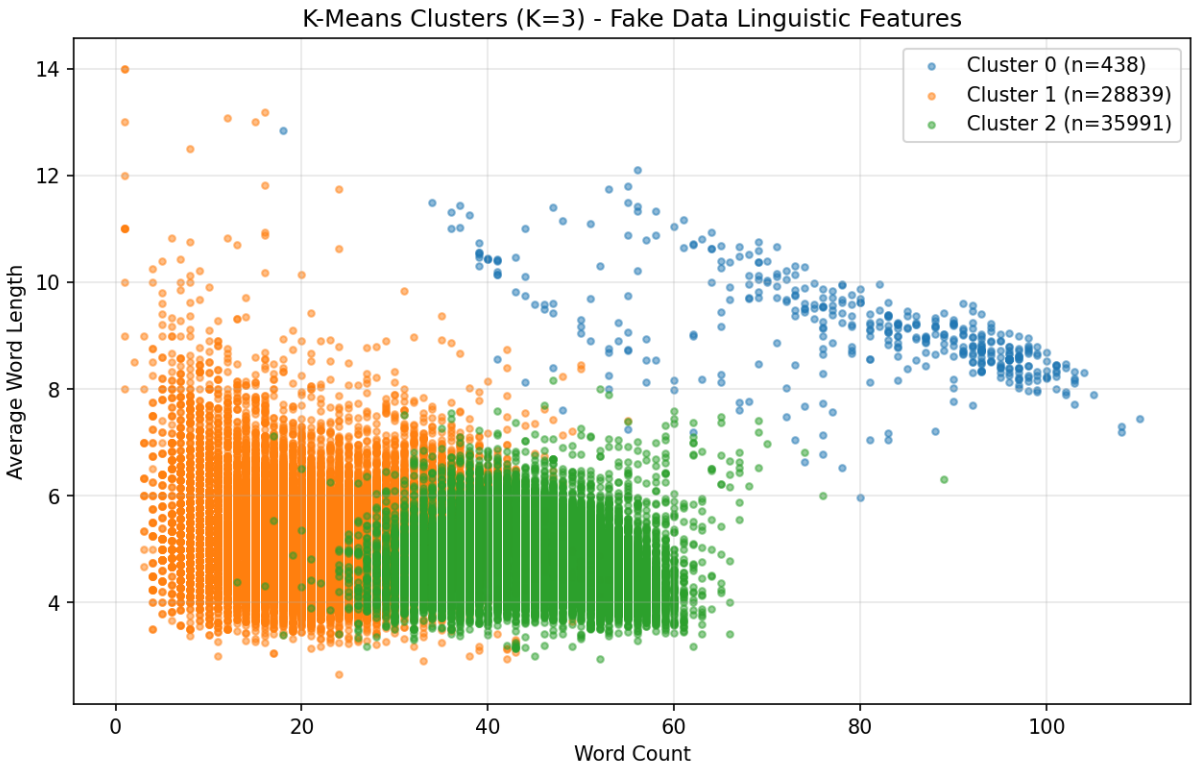


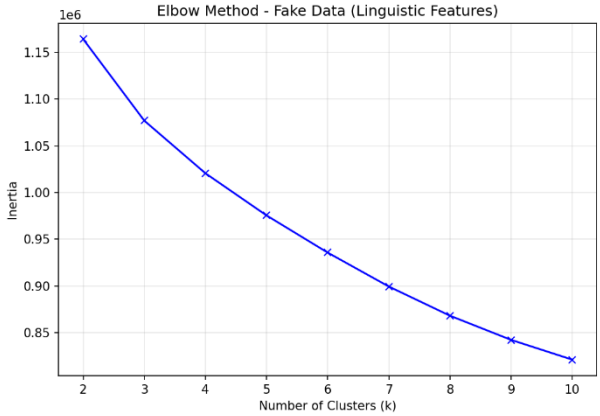

Elbow Method - All Data (Linguistic Features)
1e6
Inertia
Number of Clusters (k)
Silhouette Score - All Data (Linguistic Features)
Silhouette Score
Number of Clusters (k)
K-Means (K=3) - Fake Data
Cluster 0
Cluster 1
Cluster 2
Average Word Length
Word Count
Hierarchical (K=3) - Fake Data
Cluster 0
Cluster 1
Cluster 2
Average Word Length
Word Count

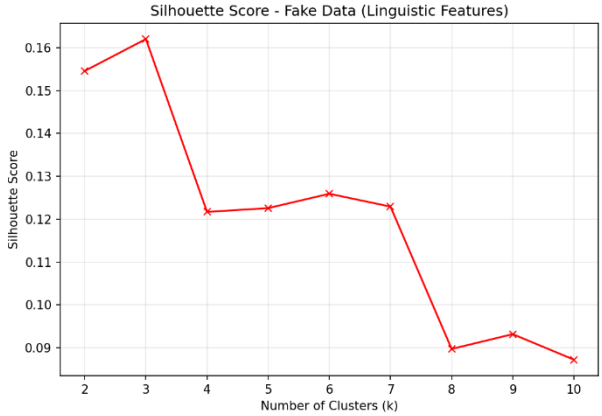

Hierarchical Clusters (K=3) - Fake Data Linguistic Features
Cluster 0 (n=6983)
Cluster 1 (n=2953)
Cluster 2 (n=64)
Average Word Length
Word Count


Dendrogram - Fake Data (Linguistic Features)
Euclidean Distance
0
50
100
150
200
Data Points


Merge Distances - Fake Data (Hierarchical Clustering)
Merge Distance
25
50
75
100
125
150
175
200
225
0
10
20
30
40
50
Number of Merges


K-Means Clusters (K=3) - Fake Data Linguistic Features
Cluster 0 (n=438)
Cluster 1 (n=28839)
Cluster 2 (n=35991)
Average Word Length
Word Count
14
12
10
8
6
4
0
20
40
60
80
100


Elbow Method - Fake Data (Linguistic Features)
1e6
Inertia
Number of Clusters (k)
1.15
1.10
1.05
1.00
0.95
0.90
0.85
2
3
4
5
6
7
8
9
10


Silhouette Score - Fake Data (Linguistic Features)
Silhouette Score
Number of Clusters (k)
0.16
0.15
0.14
0.13
0.12
0.11
0.10
0.09
2
3
4
5
6
7
8
9
10